From GGUF to Native MLX: A Reproducible Fable Migration and the Memory Policy That Blocked It

Chengyi Xu and Nexora
Nexora Research

October 2026 — experimental deployment report

Abstract

A local coding model is only usable if it can be loaded, served to a real agent and kept alive on the workstation that owns it. We report a complete migration of DavidAU's Qwen3.8-27B TURBO-Fable Cold-Fusion checkpoint from a GGUF/llama.cpp path that completed repairs but ran at 4.32–8.38 visible tokens/s to a native MLX deployment that is now serving. Three findings are new relative to our earlier Coder390/Fable study. First, a source-derived MLX affine 4-bit conversion can be built and fully verified on the target host without loading a 55 GB checkpoint into memory: a bounded 128-row CPU pipeline peaked at 3.08 GB resident memory, preserved every one of 686 non-quantized tensors bit-exactly, and matched official quantization byte-for-byte on sampled rows of six real modules. Second, native speculative decoding does not automatically help a dense 27B hybrid on this hardware. Across four interleaved conditions on an 8,159-token synthetic prompt, DFlash2 reached 8.42 tokens/s against 5.34 for plain decoding (1.57×) in the best condition, but a newer kernel generation and a wider prefill both erased the gain, and stock MLX-VLM MTP produced 6.05 against 5.08. No configuration reached the 20 tokens/s floor requested. Third, and most consequentially for operators, the deployment failure that mattered was not the model: a serving-runtime system-memory guard refused an approximately 25k-token agent prefill because macOS already held 16.7 GiB compressed while free pages sat under a 1.7 GiB floor. Disabling that documented guard, leaving the 36 GiB engine budget in place, converted the identical 25,676-token request from a refusal into a completed reply in 359.85 s. We publish the conversion receipt, per-condition measurements, both repair timeouts, the memory arithmetic and a twelve-check protocol matrix across OpenAI Chat Completions, OpenAI Responses and Anthropic Messages on both the native backend and the Magpie gateway. All quantified claims are deployment observations on one host; we make no peer-review, ranked-intelligence or near-limit-context claim.

1 Introduction

Our earlier study concluded that two proposed replacements for a working Ornith deployment were slower than the installed baselines, and recommended keeping the existing pair [1]. That conclusion was about task latency, not model quality. It also left a specific engineering question unanswered: if an operator accepts the measured latency and wants a given checkpoint anyway, can it be delivered as a verifiable native artifact on the same workstation, and what actually stops it from serving?

This report answers that question for the Fable lineage. We do not revisit the replacement comparison and we do not rank models. We describe the pipeline that produced a working deployment, the acceleration measurements that bounded what it could achieve, the memory-policy failure that blocked real agent traffic, and the evidence we retain so that a reader can check each number.

The work is deliberately narrow. One host, one checkpoint family, one runtime, one agent client. Its value is reproducibility and honesty about failure modes, not generality.

2 Artifact under study

2.1 Requested lineage and the export actually used

The operator's requested repository is DavidAU's Qwen3.8-27B TURBO-Fable Cold-Fusion 735/882 Heretic Uncensored NEO-CODER MAX MTP GGUF release, pinned at revision `ceb55042` [2]. Its card describes a multi-stage fine-tune and merge with Heretic ARA lineage, reduced reasoning-token claims and an embedded MTP head. The card's quoted 75–90 tokens/s uses an RTX 5090 under Windows and LM Studio; it is not Metal throughput and we do not reproduce it.

The deployed artifact is the `oQ4e-mtp` mixed-precision MLX export of the same BF16 parent, pinned at revision `0542355a` [3]. It is not a base-Qwen substitute. Its quantization is mixed rather than uniform: a 4-bit group-64 default with 110 five-bit and two six-bit module overrides. A separate six-bit export from the same exporter was also downloaded and tested; it is not deployed.

A prior Coder390 investigation produced a Qwen3.8-27B Coder390 EfficientThink conversion. Its measurements are included here only where they establish a runtime property, and its payloads were retired.

2.2 Why a source conversion rather than a GGUF re-quantization

The existing GGUF uses mixed tensor types: Q6_K 144, F32 360, Q4_K 108, Q8_0 50, BF16 96, Q5_K 100 and Q4_0 8 in one inspected checkpoint. Dequantizing that and requantizing to MLX affine would create a new numerical object whose relationship to the published BF16 parent is unmeasured. We therefore converted from the BF16 source shards at pinned revision `a3a1170a` for the Coder390 study, and for Fable used an exporter that states the same BF16 parent. In both cases the conversion is a declared new quantization condition, not an equivalence claim.

3 Conversion: bounded, CPU-only, fully verified

3.1 The memory problem

The Coder390 BF16 source is two shards of 49,825,162,976 and 4,888,445,168 bytes plus a 849,400,424-byte MTP sidecar. A stock converter constructs the whole model and loads every shard into memory. On a 64 GiB host that is not a bounded operation, and on a workstation with other agents running it is not a safe one.

We therefore wrote a pipeline that never instantiates a model. It verifies each source shard against a pinned size and full SHA-256, memory-maps one tensor at a time, processes at most 128 rows, applies the loader's own sanitization rules, quantizes, and appends payloads to sharded safetensors files. Peak resident memory was 3,082,305,536 bytes; the process took 493.33 s of conversion body time and 523.97 s including source rehashing.

3.2 Coupling to the loader rather than reimplementing it

Layout rules such as the depthwise-convolution axis change, the Conv3d patch-embedding transpose, and the RMSNorm offset convention are properties of the serving loader. Reimplementing them would have created a second definition of the same fact. Instead the pipeline imports a specific upstream `qwen3_5` sanitizer file, pins its SHA-256, and refuses to run if the file changes. Tensor names that the sanitizer does not recognize are rejected rather than passed through.

This mattered in practice. An early Fable export embedded its MTP tensors in the trunk shards. The loader treats the mere presence of MTP keys as evidence of raw-weight conventions and adds 1.0 to already-normalized trunk RMSNorm gains; we reproduced that shift on a small CPU tensor. Serving

that checkpoint produced incoherent text and zero draft acceptance. Separating the MTP tensors into a sidecar, without altering any tensor payload byte, restored correct output and high acceptance. The separation step repacks payloads and re-hashes each one; it performs no normalization, quantization or reordering.

3.3 What the receipts establish

The retained conversion receipt records 39 output files, 31 safetensors shards, 2,180 output tensors and a 16,054,262,240-byte payload. Independent checks confirmed index, header and payload agreement; all 686 non-quantized tensors matched the pinned sanitizer element by element, including BF16 vision weights; and packed weights, scales and biases for sampled rows of six real modules were exactly equal to official quantization output. An official module-shape construction on CPU matched all 2,180 names with no missing or extra tensors. Tokenizer parity held on five text cases and the thinking-low chat template was identical.

These checks establish layout fidelity. They do not establish model quality, which we did not measure on this artifact.

3.4 A CPU arithmetic caveat we keep

Our first arithmetic probe compared a CPU BF16 quantized matmul against a float32 reference and failed its gate at 0.048 absolute error, about 8.2–12.4% of output peak on synthetic cancellation-heavy inputs. We did not widen the tolerance. With matched float32 accumulation on both sides the packed layout matched exactly. The difference is a property of CPU BF16 arithmetic on those inputs, not evidence about GPU inference, and GPU numerical behaviour on this artifact remains unmeasured.

4 Acceleration measurements

4.1 Protocol

All synthetic comparisons use one 8,159-token prompt, greedy decoding, thinking at low effort, an unquantized KV cache and a 512-token output cap, except the condition that deliberately raises prefill width. Each condition runs four arms in the order plain, speculative, speculative, plain, so that thermal or cache drift affects both arms. Every arm in every condition produced the same 304 greedy tokens, which we report as token parity.

4.2 Results

Table 1 lists the medians. The plain and speculative columns are medians over the two arms of each kind; the request column is the whole request, prefill included.

Table 1: Synthetic 8,159-token conditions. Medians of two arms each. Decode excludes prefill; request includes it.

Condition	Plain decode	Spec. decode	Plain req.	Spec. req.
mlx-vlm 0.6.15, source MTP depth 3	5.08	6.05	192.5	180.6
mlx-dspark 0.19.0, DFlash2, prefill 512	5.34	**8.40**	180.0	**156.4**
mlx-dspark 0.20.3, newer kernels	6.50	5.89	145.8	185.3
mlx-dspark 0.19.0, DFlash2, prefill 2048	3.81	7.63	204.4	161.9

Decode is reported in tokens per second and request in seconds. The best speculative result, 8.40 tokens/s, is a 1.57× decode improvement over its paired plain arm and about 13% on the whole request, because prefill dominates an 8k prompt on this host. The newer kernel generation accepted longer drafts, 3.66 committed tokens per round against 2.43, and was nevertheless slower in both arms; longer drafts did not buy net time. Raising prefille width to 2,048 rows left speculative decode in the same band while reducing plain decode.

No condition reached 8.5 tokens/s, and none approached the 20 tokens/s target that motivated the work. With two observations per arm, balanced ordering but unpaired conditions, and a workstation under variable load, we treat these as descriptive measurements of configurations rather than as a ranking of runtimes. A related diagnostic outside this paper's table failed during CPU/GPU co-prefill calibration with a Metal command-buffer timeout at the same 8k size; that failure is retained in the artifact and the watchdog was not weakened.

4.3 Actual agent traffic

Two full Pi sessions through the Magpie gateway, each with ordinary tool and extension discovery and the same injected retry-cap defect in a fresh clone of a real client repository, are summarised in Table 2.

Table 2: Actual Pi repair sessions with native speculative decoding. Both timed out; neither repaired the defect.

	Source MTP depth 3	DFlash2
Initial input tokens	25,778	25,778
First-turn wait (s)	481.8	516.5
Whole task (s)	900.9	901.0
Tool calls / errors	4 / 0	6 / 0
Independent gate	fail	fail
Full suite	1 failed	1 failed

The gate and suite outcomes are stated as counts because both runs left the injected defect in place: one pre-existing unit test fails, 272 pass and one is skipped in each. Backend telemetry from the first run shows completed-turn decode of 7.28, 5.90, 4.83 and 3.27 tokens/s; the second reports an aggregate 3.7 tokens/s with a mean of 1.83 committed tokens per round. A 11822 tokens/s progress line in the first run's log is a buffered burst, not a decode rate, and is not used.

A short 74-token smoke test on the same artifact reached 27.40–28.48 tokens/s plain and 35.25 tokens/s with trained MTP at 2.79 accepted tokens per round. The contrast with 5–8 tokens/s at 8k is the point: short-context figures say very little about agent workloads, and we do not present them as such.

5 The failure that actually mattered

5.1 Observed refusal

After deployment, an otherwise ordinary agent turn was refused before inference. The runtime reported that the prefill stopped at a chunk boundary because macOS already held 16.7 GiB compressed while free pages were at 0.6 GiB, below the guard's 1.7 GiB floor. The engine shed caches and stayed up. We verified the arithmetic independently: a 64 GiB host with a quarter-of-RAM compressor line at 16 GiB, roughly 9.6 GiB of available supply, 1.47 GiB of immediately free pages and 2.41 GiB of derived floor.

5.2 Why the guard fired

Reading the runtime's own policy source, the compressor-full test returns true when compressed memory exceeds a quarter of RAM, free pages are under the abort floor, and free pages are also under the kernel's own target. It does not consult the reclaimable file-backed pages that its sibling thrashing test does credit. In our reading this is a deliberately conservative line calibrated against recorded freezes on larger machines, not a bug; the practical consequence is that a 64 GiB desktop which legitimately keeps 16 GiB compressed can be refused while several gigabytes remain reclaimable.

5.3 The supported change

The runtime exposes a documented switch that disables the system-memory prefill guard, distinct from its engine memory limit. We set it in the service process only, verified the effective value inside the live native child, restarted, and re-issued the identical request. The same 25,676-token input then completed in 359.85 s with the exact expected marker, with no tools, extensions, prompts or budgets reduced. The alternate explanation, that the second attempt merely benefited from a quieter machine, is weakened by the identical input size and by the measured setting, but cannot be excluded from a single observation.

This is an operator-authorised trade. With the system guard disabled, the engine's 36 GiB allocator budget remains the binding limit; genuine allocator failures are still reported. We did not disable the allocator limit, enable swap forcing, change OS power settings or alter fan control.

6 Deployment and protocol verification

The deployed profile serves native 262,144-token context at MTP depth 3 under the runtime's turbo profile with a 128-row prefill, Q8 KV cache, temperature 0.6 and medium default reasoning effort. Transcript rewriting is explicitly off and we confirmed zero injective, compacting or replacing transcript counters on the measured requests. A separate private credential is used, service logs are filtered so that browser-auth startup URLs cannot leak it, and the wrapper terminates its own native child on shutdown.

Table 3 records the interface matrix. Both the native backend and the Magpie gateway answered all three dialects, streaming and non-streaming, with exact markers and terminal stream events.

Table 3: Protocol verification. Twelve checks, all passing.

Dialect	Native non-stream	Native stream	Gateway (both)
OpenAI Chat Completions	pass	pass	pass
OpenAI Responses	pass	pass	pass
Anthropic Messages	pass	pass	pass

Because the change is a service-level setting behind a shared endpoint, every harness that speaks these routes inherits it without client configuration. We verified actual Pi traffic and the three documented dialects; we did not test every third-party client's optional features.

Operationally, the swap replaced the previous abliterated slot. Normal Ornith, the image service and the shared draft were preserved and re-verified: normal Ornith answered a live request, and an image-service handoff started the image backend and restored the text backend before returning. The retired checkpoint and its owned prepared cache were deleted only after the new profile reported ready, the retired file's full hash matched its recorded identity, and no process held it open; 42,386,495,264

logical bytes were released across 43 entries. Logical bytes are not a claim of equivalent physical reclamation under APFS.

7 Interpretation and limitations

What is established. A source-derived mixed-precision MLX artifact can be produced and verified on a 64 GiB host without materializing the checkpoint in memory; speculative decoding gives a real but modest and configuration-sensitive improvement on synthetic 8k traffic; and a serving-runtime memory policy, not the model, converted a usable deployment into a refusing one, with a documented switch that resolves it.

What is not established. We make no claim that Fable is more or less capable than any other model. We did not measure reasoning or coding accuracy beyond the pass/fail of one injected-defect task, which failed. We did not qualify near-limit context: 262,144 is allocation metadata here and the largest actual request recorded is 32,014 tokens. We did not reach 20 tokens/s in any configuration and do not claim a latency floor. We did not compare runtime versions as a controlled experiment; each condition ran twice, unpaired, on a shared workstation with reported swap and background load.

Threats to validity. Host contention, page-cache state and GPU thermal drift all vary between conditions. The memory-guard comparison rests on one before-and-after pair at matched input size. The deployed quantization is a new numerical object whose agreement with the BF16 parent we measured only through layout and sampling, not through task accuracy. Open-weight declarations such as "uncensored" or "Heretic" are treated as provenance labels; no refusal-removal or harmful-generation evaluation was performed, and nothing here endorses such use.

8 Artifacts and reproducibility

The public artifact contains the sanitized evidence document, the export and validation code, the protocol and licence. The exporter is whitelist-only: it reads finite scalars and enumerated flags, rejects any string carrying a private path or credential marker, and refuses to write an artifact whose serialized form contains one. Prompts, tool output, repository source, agent transcripts and local absolute paths are excluded. The synthetic prompt is described, not published; the agent task is described by its injected defect and gate, both of which are reproducible from the protocol.

9 Conclusion

The interesting result of this migration is not a speed number. It is that a rigorous conversion effort, a careful speculative-decoding comparison and a documented, reversible runtime policy change were each necessary before a checkpoint that had already been downloaded and tested would serve real agent traffic. Reporting only the conversion would have implied a working deployment; reporting only the timeouts would have implied an unusable model. Both the success and the refusal belong in the record.

References

[1] C. Xu and Nexora, "Fast on the Card, Slow in the Agent: A Vision and Speculation Study of Coder390 and TURBO-Fable on Apple Silicon," Nexora Research, 2026.

https://huggingface.co/datasets/ChengyiX/

[2] DavidAU, "Qwen3.8-27B-TURBO-Fable-Cold-Fusion-735-882-Heretic-Uncensored-NEO-CODER-MAX-MTP-GGUF," Hugging Face, revision `ceb55042`.
https://huggingface.co/DavidAU/

[3] zviratko, "Qwen3.8-27B TURBO-Fable oQ4e-mtp MLX export," Hugging Face, revision `0542355a`; full repository id recorded in the evidence artifact.
https://huggingface.co/zviratko/

[4] Y. Altoukhi, "MTPLX: native MTP speculative decoding on Apple Silicon," revision `9882703f`.
https://github.com/youssofal/MTPLX

[5] A. Rahim, "mlx-dspark: DSpark and DFlash speculative decoding for MLX," revisions `d6042f38` and `0.19.0`. https://github.com/ARahim3/mlx-dspark

[6] Blaizzy et al., "mlx-vlm," version 0.6.15. https://github.com/Blaizzy/mlx-vlm

[7] SGLang, "Apple Metal platform notes" and `speculative_hook.py` at `72eab59b`; in-tree DFlash excludes MPS. https://docs.sglang.io/docs/hardware-platforms/apple_metal

[8] DeepSeek-AI, "DeepSeek-V4.1-Flash-1017," 763B-parameter MoE with 1M context. Inspected for size feasibility only; not runnable on a 64 GiB host.
https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017